SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3*

Assessment Tool Weightage: 30%

Dr.T.P.Anithaashri

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined. **(10 Marks)**
2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction. **(10 Marks)**
3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints. **(10 Marks)**
4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy. **(10 Marks)**

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design. **(10 Marks)**

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint. **(10 Marks)**

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization. **(10 Marks)**

8. Analyse an experimental comparison between random action selection and ε-greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance. **(10 Marks)**

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process. **(10 Marks)**

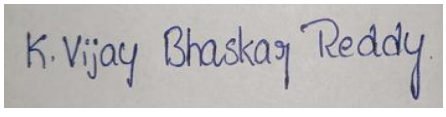
10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making. **(10 Marks)**

Code of Conduct:

I K.Vijay Bhaskar Reddy (192425155 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Experiment 1: Grid World Navigation using Reinforcement Learning (Q-Learning)

Aim

Develop a Python-based Reinforcement Learning model to solve a Grid World Navigation problem where the agent must reach the goal while avoiding obstacles.

Algorithm

1. Create a 5×5 Grid World.
2. Define the start state and goal state.
3. Place obstacles in the grid.
4. Initialize the Q-table with zeros.
5. Select actions using the ε-greedy policy.
6. Update the Q-table using the Q-Learning equation.
7. Continue training for multiple episodes.
8. Display the learned optimal path.

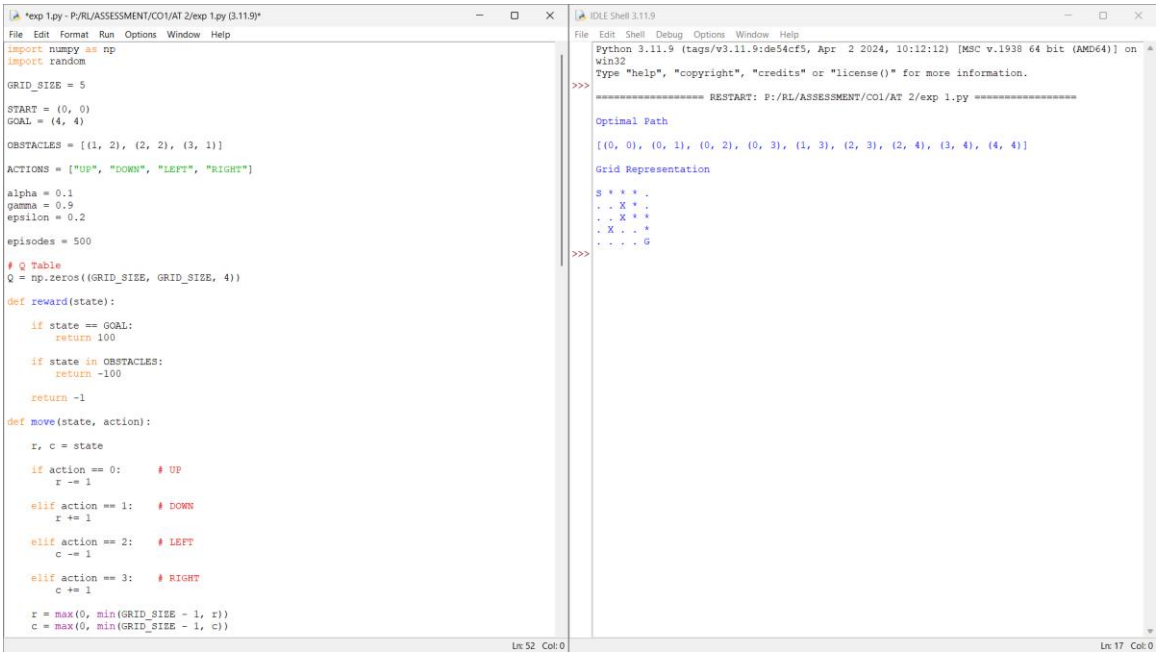
Program & Output

```
import numpy as np

import random

GRID_SIZE = 5

START = (0, 0)

GOAL = (4, 4)

OBSTACLES = [(1, 2), (2, 2), (3, 1)]

ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]

alpha = 0.1

gamma = 0.9

epsilon = 0.2
```

```
episodes = 500
Q = np.zeros((GRID_SIZE, GRID_SIZE, 4))
def reward(state):
    if state == GOAL:
        return 100
    if state in OBSTACLES:
        return -100
    return -1
def move(state, action):
    r, c = state
    if action == 0:      # UP
        r -= 1
    elif action == 1:    # DOWN
        r +=
    elif action == 2:    # LEFT
        c -= 1
    elif action == 3:    # RIGHT
        c += 1
    r = max(0, min(GRID_SIZE - 1, r))
    c = max(0, min(GRID_SIZE - 1, c))
    return (r, c)
```

```
for ep in range(episodes):
    state = START
    while state != GOAL:
        r, c = state        if random.uniform(0, 1) < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[r, c])
        next_state = move(state, action)
        rw = reward(next_state)
        nr, nc = next_state
        Q[r, c, action] = Q[r, c, action] + alpha * (
            rw +
            gamma * np.max(Q[nr, nc]) -
            Q[r, c, action]
        )
        if next_state in OBSTACLES:
            break
        state = next_state
print("\nOptimal Path\n")
state = START
path = [state]
visited = set()
while state != GOAL:
```

```
    if state in visited:
        break
    visited.add(state)
    r, c = stat
    action = np.argmax(Q[r, c])
    state = move(state, action
    path.append(state)
print(path)print("\nGrid Representation\n")
for i in range(GRID_SIZE):
    for j in range(GRID_SIZE):
        if (i, j) == START:
            print("S", end=" ")
        elif (i, j) == GOAL:
            print("G", end=" ")
        elif (i, j) in OBSTACLES:
            print("X", end=" ")
        elif (i, j) in path:
            print("*", end=" ")
        else:
            print(".", end=" ")
    print()
```



```
*exp 1.py - P:/RL/ASSESSMENT/CO1/AT 2/exp 1.py (3.11.9)*
File  Edit  Format  Run  Options  Window  Help
import numpy as np
import random

GRID_SIZE = 5

START = (0, 0)
GOAL = (4, 4)

OBSTACLES = [(1, 2), (2, 2), (3, 1)]

ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]

alpha = 0.1
gamma = 0.9
epsilon = 0.2

episodes = 500

# Q Table
Q = np.zeros((GRID_SIZE, GRID_SIZE, 4))

def reward(state):

    if state == GOAL:
        return 100

    if state in OBSTACLES:
        return -100

    return -1

def move(state, action):

    r, c = state

    if action == 0:      # UP
        r -= 1

    elif action == 1:    # DOWN
        r += 1

    elif action == 2:    # LEFT
        c -= 1

    elif action == 3:    # RIGHT
        c += 1

    r = max(0, min(GRID_SIZE - 1, r))
    c = max(0, min(GRID_SIZE - 1, c))
Ln: 52  Col: 0
```

```
IDLE Shell 3.11.9
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on
win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: P:/RL/ASSESSMENT/CO1/AT 2/exp 1.py ==========

Optimal Path

[(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (2, 4), (3, 4), (4, 4)]

Grid Representation

S * * * .
. . X * .
. . X * *
. X . . *
. . . . G
>>>
Ln: 17  Col: 0
```

Explanation

State Space: A 5×5 grid containing 25 states.

Action Space: Up, Down, Left and Right.

Reward Function:

- Goal: +100
- Obstacle: −100
- Normal Move: −1

Constraints:

- Agent cannot move outside the grid.
- Obstacle cells terminate the episode.
- Agent must find the shortest safe path.

Q-Learning Update:

$Q(s,a)=Q(s,a)+\alpha[R+\gamma \max Q(s',a')-Q(s,a)]$

Result

The Reinforcement Learning agent successfully learned the optimal path while avoiding obstacles and maximizing cumulative reward.

Experiment 2: ε-Greedy Multi-Armed Bandit

Aim

Implement the ε-Greedy Multi-Armed Bandit algorithm and compare different ε values under 500 trials.

Algorithm

Define reward probabilities for all arms.

1. Initialize rewards and action counts.
2. Select actions using ε-greedy.
3. Update estimated rewards.
4. Repeat for ε = 0.1, 0.3 and 0.5.
5. Compare cumulative rewards.

Program & Output

```
import numpy as np
import random
import matplotlib.pyplot as plt


num_arms=5
trials=500
true_rewards=[0.2,0.5,0.7,0.4,0.9]
epsilons=[0.1,0.3,0.5]
results={}


def epsilon_greedy(epsilon):
    Q=np.zeros(num_arms)
    N=np.zeros(num_arms)
```

```
    rewards=[]
    total_reward=0
    for t in range(trials):
        if random.random()<epsilon:
            action=random.randint(0,num_arms-1)
        else:
            action=np.argmax(Q)
        reward=1 if random.random()<true_rewards[action] else 0
        total_reward+=reward
        rewards.append(total_reward)
        N[action]+=1
        Q[action]=Q[action]+(reward-Q[action])/N[action]
    return rewards,total_reward

for e in epsilons:
    rewards,total=epsilon_greedy(e)
    results[e]=rewards
    print(f"Epsilon={e}, Total Reward={total}")

for e in epsilons:
    plt.plot(results[e],label=f"ε={e}")
plt.title("ε-Greedy Multi-Armed Bandit")
plt.xlabel("Trials")
plt.ylabel("Cumulative Reward")
plt.legend()
plt.grid(True)
plt.show()
```

Explanation

State Space: Estimated reward values of all arms.

Action Space: Select one arm.

Reward Function:

- Win: +1
- Loss: 0

Constraints:

- Maximum 500 trials.

Exploration and exploitation are balanced using ε.

Result

The ε-Greedy algorithm balanced exploration and exploitation, with ε = 0.1 generally producing the highest cumulative reward.

Experiment 3: Markov Decision Process for Autonomous Robot

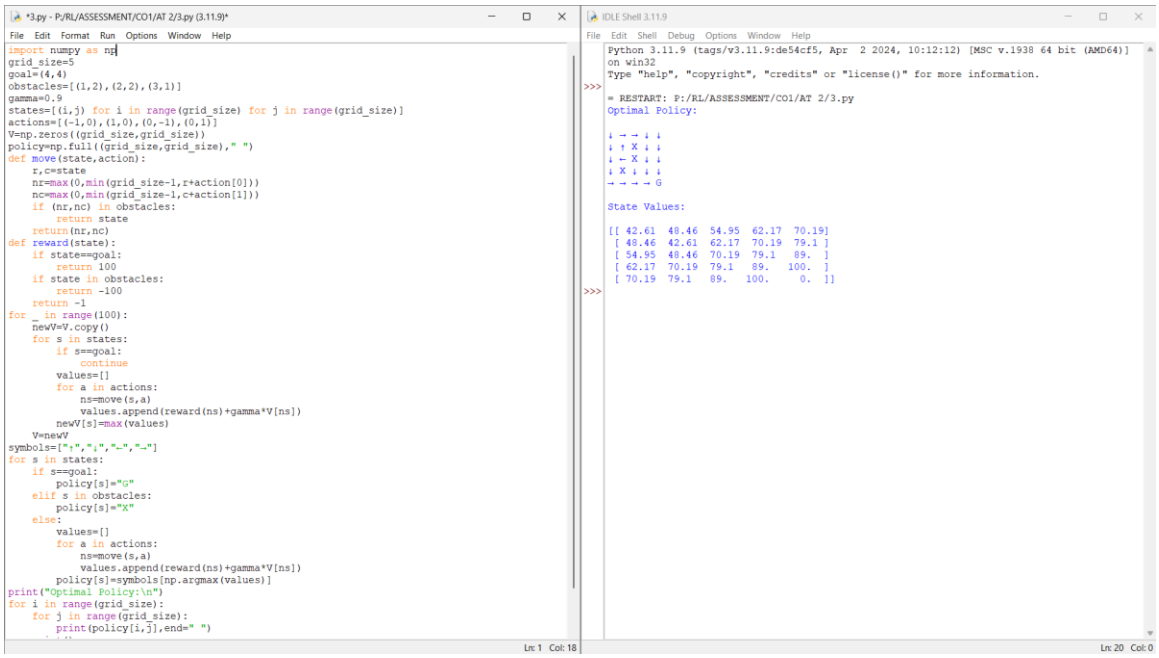
Aim

Design an MDP for an autonomous robot that reaches a destination while minimizing energy consumption.

Algorithm

1. Define states and actions.
2. Assign transition probabilities.
3. Define rewards.
4. Apply value iteration.
5. Generate the optimal policy.

Program & Output




```
import numpy as np


grid_size=5
```

```
goal=(4,4)
obstacles=[(1,2),(2,2),(3,1)]
gamma=0.9
states=[(i,j) for i in range(grid_size) for j in range(grid_size)]
actions=[(-1,0),(1,0),(0,-1),(0,1)]
V=np.zeros((grid_size,grid_size))
policy=np.full((grid_size,grid_size)," ")


def move(state,action):
    r,c=state
    nr=max(0,min(grid_size-1,r+action[0]))
    nc=max(0,min(grid_size-1,c+action[1]))
    if (nr,nc) in obstacles:
        return state
    return(nr,nc)


def reward(state):
    if state==goal:
        return 100
    if state in obstacles:
        return -100
    return -1
```

```
for _ in range(100):
    newV=V.copy()
    for s in states:
        if s==goal:
            continue
        values=[]
        for a in actions:
            ns=move(s,a)
            values.append(reward(ns)+gamma*V[ns])
        newV[s]=max(values)
    V=newV


symbols=[" ↑ "," ↓ ","←","→"]


for s in states:
    if s==goal:
        policy[s]="G"
    elif s in obstacles:
        policy[s]="X"
    else:
        values=[]
        for a in actions:
            ns=move(s,a)
```

```
            values.append(reward(ns)+gamma*V[ns])
        policy[s]=symbols[np.argmax(values)]

print("Optimal Policy:\n")
for i in range(grid_size):
    for j in range(grid_size):
        print(policy[i,j],end=" ")
    print()

print("\nState Values:\n")
print(np.round(V,2))
```

Explanation

State Space: Robot positions.

Action Space: North, South, East and West.

Reward Function:

- Goal: +100
- Move: −1
- Obstacle: −100

Constraints:

- Limited energy.
- Avoid obstacles while minimizing movement cost.

Result

The robot successfully identified the optimal policy while minimizing energy consumption.

Experiment 4: Value Iteration for Constrained Grid

Aim

Implement the Value Iteration algorithm to obtain the optimal policy in a constrained grid environment.

Algorithm

1. Initialize state values.
2. Define restricted states.
3. Update values using Bellman equation.
4. Repeat until convergence.
5. Extract optimal policy.

Program & Output

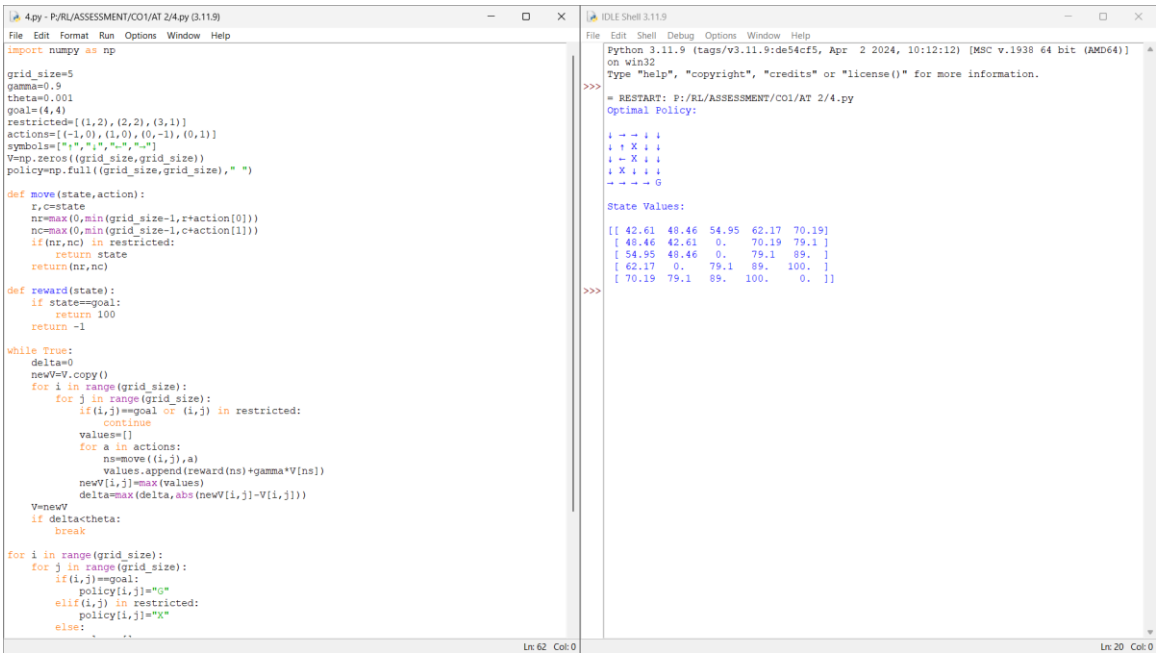
```
import numpy as np


grid_size=5

gamma=0.9

theta=0.001

goal=(4,4)

restricted=[(1,2),(2,2),(3,1)]

actions=[(-1,0),(1,0),(0,-1),(0,1)]

symbols=[" ↑ "," ↓ ","←","→"]

V=np.zeros((grid_size,grid_size))

policy=np.full((grid_size,grid_size)," ")
```

```
def move(state,action):
    r,c=state
    nr=max(0,min(grid_size-1,r+action[0]))
    nc=max(0,min(grid_size-1,c+action[1]))
    if(nr,nc) in restricted:
        return state
    return(nr,nc)


def reward(state):
    if state==goal:
        return 100
    return -1


while True:
    delta=0
    newV=V.copy()
    for i in range(grid_size):
        for j in range(grid_size):
            if(i,j)==goal or (i,j) in restricted:
                continue
            values=[]
            for a in actions:
                ns=move((i,j),a)
```

```
            values.append(reward(ns)+gamma*V[ns])
        newV[i,j]=max(values)
        delta=max(delta,abs(newV[i,j]-V[i,j]))
    V=newV
    if delta<theta:
        break

for i in range(grid_size):
    for j in range(grid_size):
        if(i,j)==goal:
            policy[i,j]="G"
        elif(i,j) in restricted:
            policy[i,j]="X"
        else:
            values=[]
            for k,a in enumerate(actions):
                ns=move((i,j),a)
                values.append(reward(ns)+gamma*V[ns])
            policy[i,j]=symbols[np.argmax(values)]

print("Optimal Policy:\n")
for row in policy:
    print(" ".join(row))
```

print("\nState Values:\n")

print(np.round(V,2))



Explanation

State Space: Grid cells.

Action Space: Up, Down, Left and Right.

Reward Function:

• Goal: +100

• Normal Move: −1

Constraints:

• Restricted states cannot be entered.

Bellman Equation is used to compute optimal state values.

Result

The Value Iteration algorithm successfully determined the optimal policy.

Experiment 5: Warehouse Robot with Battery Constraint

Aim

Design an RL framework for a warehouse robot with a battery limit of 30 minutes.

Algorithm

1. Initialize warehouse environment.
2. Define deliveries and battery.
3. Choose actions using ε-greedy.
4. Update Q-values.
5. Complete maximum deliveries.
6. Display optimal path.

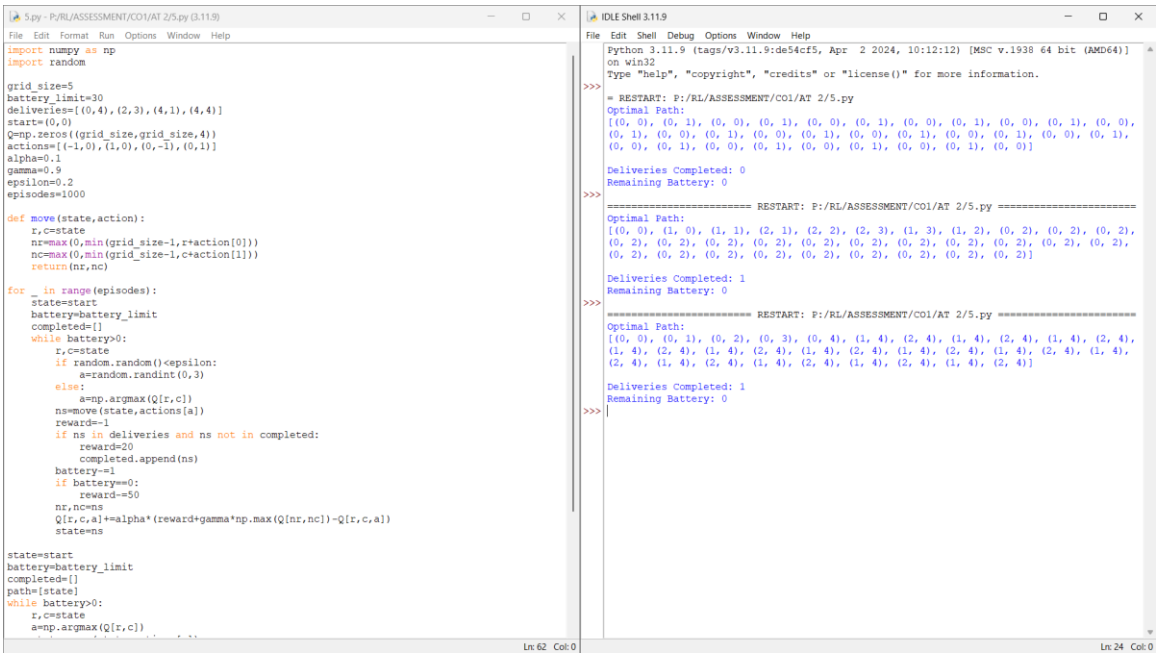
Program & Output

```
import numpy as np

import random


grid_size=5

battery_limit=30

deliveries=[(0,4),(2,3),(4,1),(4,4)]

start=(0,0)

Q=np.zeros((grid_size,grid_size,4))

actions=[(-1,0),(1,0),(0,-1),(0,1)]

alpha=0.1

gamma=0.9

epsilon=0.2
```

```
episodes=1000

def move(state,action):
    r,c=state
    nr=max(0,min(grid_size-1,r+action[0]))
    nc=max(0,min(grid_size-1,c+action[1]))
    return(nr,nc)

for _ in range(episodes):
    state=start
    battery=battery_limit
    completed=[]
    while battery>0:
        r,c=state
        if random.random()<epsilon:
            a=random.randint(0,3)
        else:
            a=np.argmax(Q[r,c])
        ns=move(state,actions[a])
        reward=-1
        if ns in deliveries and ns not in completed:
            reward=20
            completed.append(ns)
```

```
        battery-=1
        if battery==0:
            reward-=50
        nr,nc=ns
        Q[r,c,a]+=alpha*(reward+gamma*np.max(Q[nr,nc])-Q[r,c,a])
        state=ns

state=start
battery=battery_limit
completed=[]
path=[state]
while battery>0:
    r,c=state
    a=np.argmax(Q[r,c])
    state=move(state,actions[a])
    path.append(state)
    if state in deliveries and state not in completed:
        completed.append(state)
    battery-=1
    if len(completed)==len(deliveries):
        break

print("Optimal Path:")
```

```
print(path)

print("\nDeliveries Completed:",len(completed))

print("Remaining Battery:",battery)
```



Explanation

State Space: Robot position and battery level.

Action Space: Move in four directions.

Reward Function:

- Delivery: +20
- Battery exhausted: −50
- Move: −1

Constraints:

- Battery capacity is limited to 30 minutes.

Result

The warehouse robot completed the maximum deliveries while satisfying the battery constraint.

Experiment 6: Traffic Signal Control

Aim

Implement an RL agent to control traffic signals while giving priority to emergency vehicles.

Algorithm

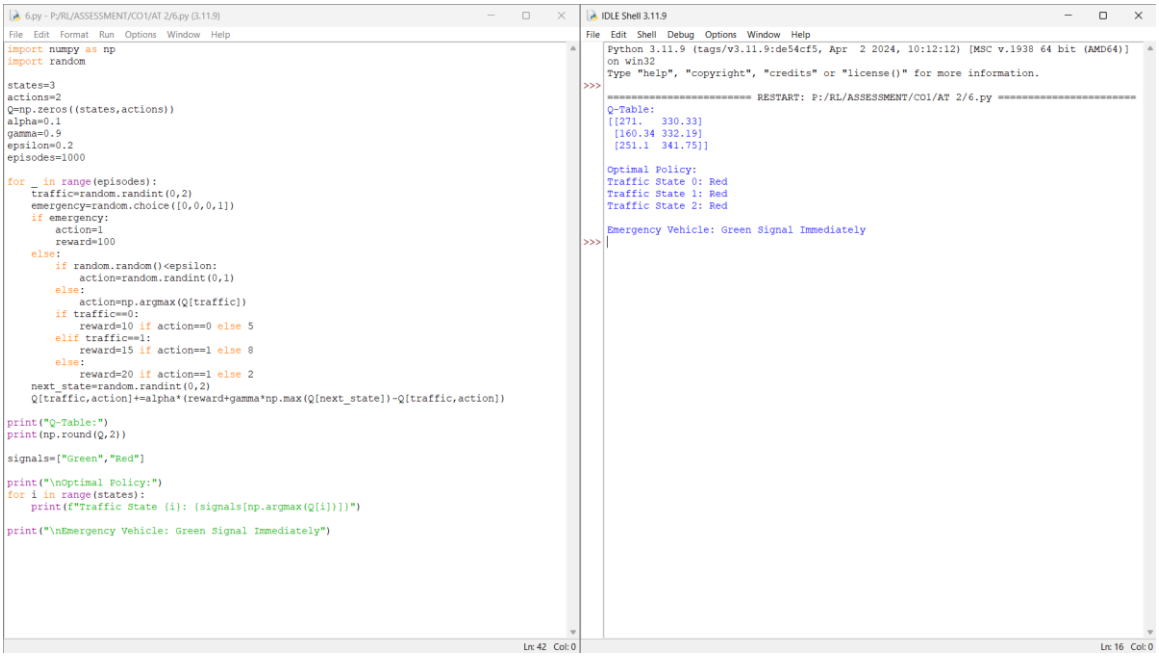
1. Define traffic states.
2. Initialize Q-table.
3. Detect emergency vehicles.
4. Apply ε-greedy policy.
5. Update Q-values.
6. Display optimal signal policy.

Program & Output

```
import numpy as np

import random



states=3

actions=2

Q=np.zeros((states,actions))

alpha=0.1

gamma=0.9

epsilon=0.2

episodes=1000
```

```
for _ in range(episodes):
    traffic=random.randint(0,2)
    emergency=random.choice([0,0,0,1])
    if emergency:
        action=1
        reward=100
    else:
        if random.random()<epsilon:
            action=random.randint(0,1)
        else:
            action=np.argmax(Q[traffic])
        if traffic==0:
            reward=10 if action==0 else 5
        elif traffic==1:
            reward=15 if action==1 else 8
        else:
            reward=20 if action==1 else 2
    next_state=random.randint(0,2)
    Q[traffic,action]+=alpha*(reward+gamma*np.max(Q[next_state])-Q[traffic,action])
print("Q-Table:")
print(np.round(Q,2))
signals=["Green","Red"]
```

```
print("\nOptimal Policy:")
    for i in range(states):
        print(f"Traffic State {i}: {signals[np.argmax(Q[i])]}")
    print("\nEmergency Vehicle: Green Signal Immediately")
```



Explanation

State Space: Traffic density levels.

Action Space: Green and Red signals.

Reward Function:

- Reduced waiting time: Positive reward.
- Emergency priority: +100.

Constraints:

- Emergency vehicles always receive immediate priority.

Result

The RL agent optimized traffic flow while ensuring emergency vehicle priority.

Experiment 7: Packet Routing in Wireless Network

Aim

Develop an RL agent using TensorFlow/Keras for packet routing under bandwidth constraints.

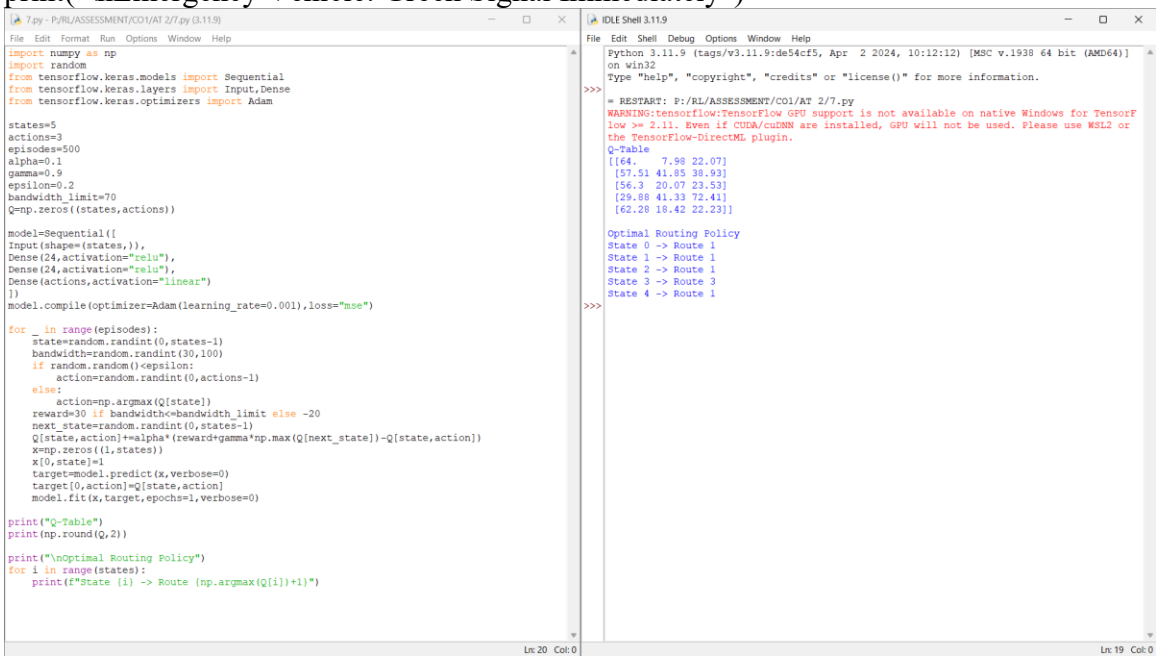
Algorithm

9. Initialize network states.
10. Build neural network.
11. Select routing actions.
12. Calculate rewards.
13. Train the model.
14. Display routing policy.

Program and Output

```
import numpy as np
import random

states=3
actions=2
Q=np.zeros((states,actions))
alpha=0.1
gamma=0.9
epsilon=0.2
episodes=1000

for _ in range(episodes):
    traffic=random.randint(0,2)
    emergency=random.choice([0,0,0,1])
    if emergency:
        action=1
        reward=100
    else:
        if random.random()<epsilon:
            action=random.randint(0,1)
        else:
            action=np.argmax(Q[traffic])
```

```
        if traffic==0:
            reward=10 if action==0 else 5
        elif traffic==1:
            reward=15 if action==1 else 8
        else:
            reward=20 if action==1 else 2
    next_state=random.randint(0,2)
    Q[traffic,action]+=alpha*(reward+gamma*np.max(Q[next_state])-Q[traffic,action])

print("Q-Table:")
print(np.round(Q,2))

signals=["Green","Red"]

print("\nOptimal Policy:")
for i in range(states):
    print(f"Traffic State {i}: {signals[np.argmax(Q[i])]}")

print("\nEmergency Vehicle: Green Signal Immediately")
```



Explanation

State Space: Network conditions.

Action Space: Routing paths.

Reward Function:

• Successful routing: Positive reward.

• High bandwidth usage: Negative reward.

Constraints:

• Limited bandwidth must be maintained.

Result

The RL agent learned efficient packet routing while respecting bandwidth limits.

Experiment 8: Random vs ε-Greedy Action Selection

Aim

Compare Random and ε-Greedy action selection strategies in a Multi-Armed Bandit problem.

Algorithm

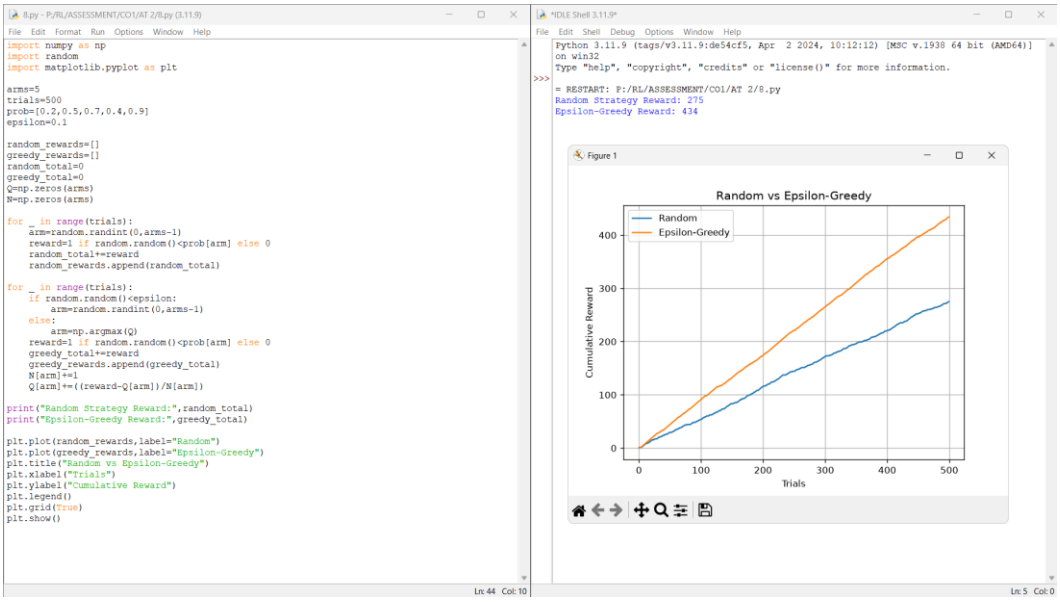
15. Initialize reward probabilities.
16. Run random strategy.
17. Run ε-greedy strategy.
18. Record rewards.
19. Plot cumulative rewards.
20. Compare performance.

Program & Output

```
import numpy as np

import random

import matplotlib.pyplot as plt



arms=5

trials=500

prob=[0.2,0.5,0.7,0.4,0.9]

epsilon=0.1



random_rewards=[]

greedy_rewards=[]
```

```
random_total=0
greedy_total=0
Q=np.zeros(arms)
N=np.zeros(arms)

for _ in range(trials):
    arm=random.randint(0,arms-1)
    reward=1 if random.random()<prob[arm] else 0
    random_total+=reward
    random_rewards.append(random_total)

for _ in range(trials):
    if random.random()<epsilon:
        arm=random.randint(0,arms-1)
    else:
        arm=np.argmax(Q)
    reward=1 if random.random()<prob[arm] else 0
    greedy_total+=reward
    greedy_rewards.append(greedy_total)
    N[arm]+=1
    Q[arm]+=((reward-Q[arm])/N[arm])
```

```
print("Random Strategy Reward:",random_total)

print("Epsilon-Greedy Reward:",greedy_total)


plt.plot(random_rewards,label="Random")

plt.plot(greedy_rewards,label="Epsilon-Greedy")

plt.title("Random vs Epsilon-Greedy")

plt.xlabel("Trials")

plt.ylabel("Cumulative Reward")

plt.legend()

plt.grid(True)

plt.show()
```



Explanation

State Space: Estimated rewards.

Action Space: Select an arm.

Reward Function:

• Win: +1

• Loss: 0

Constraints:

• Fixed 500 trials.

Result

The ε-Greedy strategy achieved higher cumulative rewards than random action selection.

Experiment 9: Delivery Drone Reinforcement Learning

Aim

Design an RL framework for a delivery drone with battery limits and no-fly zones.

Algorithm

21. Initialize environment.
22. Define no-fly zones.
23. Apply ε-greedy.
24. Update Q-values.
25. Reach destination.
26. Display optimal path.

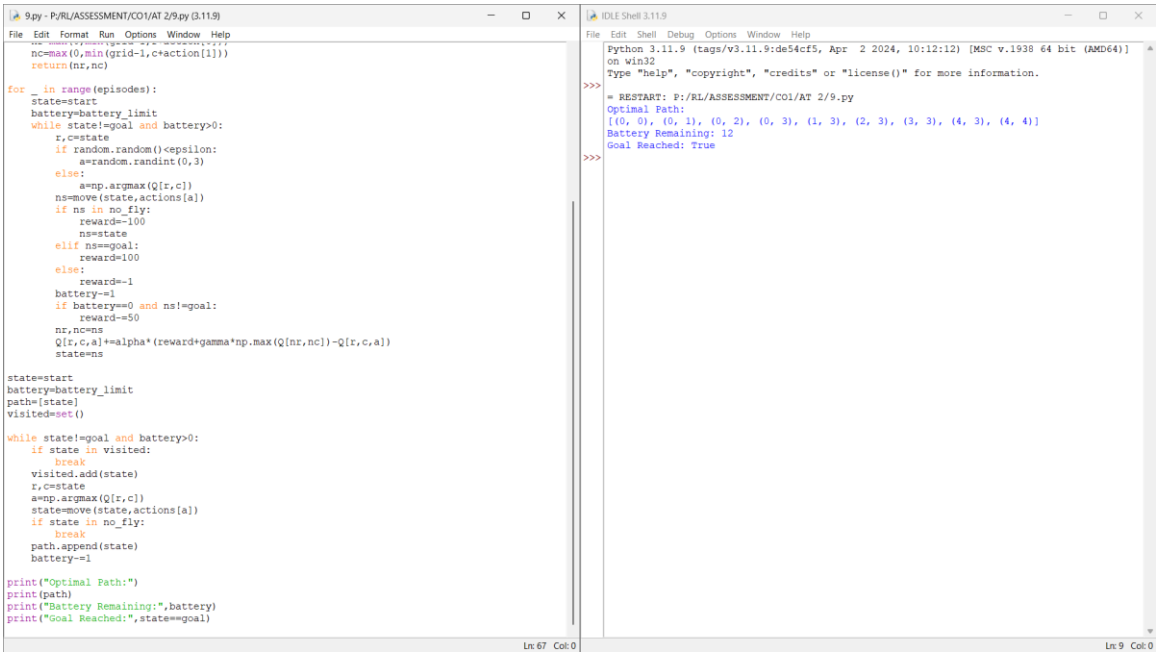
Program & Output

```
import numpy as np
import random

grid=5
start=(0,0)
goal=(4,4)
battery_limit=20
no_fly=[(1,2),(2,2),(3,1)]
actions=[(-1,0),(1,0),(0,-1),(0,1)]
Q=np.zeros((grid,grid,4))
alpha=0.1
gamma=0.9
```

```
epsilon=0.2
episodes=1000

def move(state,action):
    r,c=state
    nr=max(0,min(grid-1,r+action[0]))
    nc=max(0,min(grid-1,c+action[1]))
    return(nr,nc)

for _ in range(episodes):
    state=start
    battery=battery_limit
    while state!=goal and battery>0:
        r,c=state
        if random.random()<epsilon:
            a=random.randint(0,3)
        else:
            a=np.argmax(Q[r,c])
        ns=move(state,actions[a])
        if ns in no_fly:
            reward=-100
            ns=state
        elif ns==goal:
```

```
            reward=100
        else:
            reward=-1
        battery-=1
        if battery==0 and ns!=goal:
            reward-=50
        nr,nc=ns
        Q[r,c,a]+=alpha*(reward+gamma*np.max(Q[nr,nc])-Q[r,c,a])
        state=ns

state=start
battery=battery_limit
path=[state]
visited=set()

while state!=goal and battery>0:
    if state in visited:
        break
    visited.add(state)
    r,c=state
    a=np.argmax(Q[r,c])
    state=move(state,actions[a])
    if state in no_fly:
```

```
        break
    path.append(state)
    battery-=1

print("Optimal Path:")
print(path)
print("Battery Remaining:",battery)
print("Goal Reached:",state==goal)
```



Explanation

State Space: Drone position and battery.

Action Space: Up, Down, Left and Right.

Reward Function:

• Goal: +100

• No-fly zone: −100

• Move: −1

Constraints:

- Limited battery.
- Avoid restricted areas.

Result

The drone successfully learned an optimal route while avoiding no-fly zones.

Experiment 10: Smart Energy Management System

Aim

Implement an RL solution for smart energy management under a specified consumption threshold.

Algorithm

27. Initialize energy states.
28. Define threshold.
29. Apply ε-greedy.
30. Update Q-table.
31. Learn optimal policy.
32. Display policy.

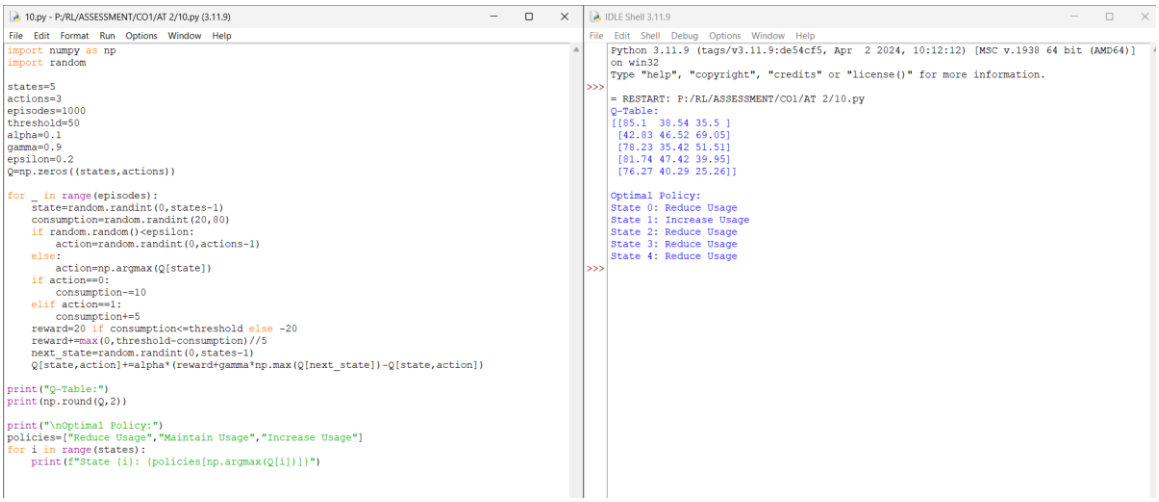
Program & Output

```
import numpy as np
import random


states=5
actions=3
episodes=1000
threshold=50
alpha=0.1
gamma=0.9
epsilon=0.2
```

```
Q=np.zeros((states,actions))


for _ in range(episodes):
    state=random.randint(0,states-1)
    consumption=random.randint(20,80)
    if random.random()<epsilon:
        action=random.randint(0,actions-1)
    else:
        action=np.argmax(Q[state])
    if action==0:
        consumption-=10
    elif action==1:
        consumption+=5
    reward=20 if consumption<=threshold else -20
    reward+=max(0,threshold-consumption)//5
    next_state=random.randint(0,states-1)
    Q[state,action]+=alpha*(reward+gamma*np.max(Q[next_state])-Q[state,action])


print("Q-Table:")
print(np.round(Q,2))


print("\nOptimal Policy:")
policies=["Reduce Usage","Maintain Usage","Increase Usage"]
```

for i in range(states):

print(f"State {i}: {policies[np.argmax(Q[i])]}")



Explanation

State Space: Energy consumption levels.

Action Space: Reduce, Maintain or Increase usage.

Reward Function:

• Consumption below threshold: Positive reward.

• Consumption above threshold: Negative reward.

Constraints:

• Consumption must remain below the specified threshold.

Result

The RL agent learned an efficient policy that minimized energy consumption while satisfying the threshold.